Laboratory Research Problem 03 (LRP03)

CMSC 180: Introduction to Parallel Computing

Charles Andrei P. De los Reyes
CD-3L

March 26, 2026

Table 1: Runtime Results for $n = 25,000$

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
25,000	1	16.882696	16.646522	16.418576	16.649265
25,000	2	14.613822	14.534789	14.820335	14.656315
25,000	4	13.796131	13.219227	13.303665	13.439674
25,000	8	10.622112	11.174998	10.216272	10.671127
25,000	16	10.129116	11.816843	10.129116	10.691692
25,000	32	11.072569	10.583262	10.456921	10.704251
25,000	64	10.381563	10.076244	10.074662	10.177490

Research Question 1

What is the difference of the average time that you obtained for $t = 1$ compared to the average that was obtained in LRP01 and LRP02?

The average time that I obtained for $t = 1$ in LRP03 is slightly slower than LRP01 and LRP02. This happens because LRP01 is a serial program with zero overhead. LRP02 introduces thread creation overhead, which is why it is slightly slower than LRP01. LRP03 is even slightly slower than LRP02 because, on top of the thread creation overhead, it also has system call overhead. The program must execute the `SetThreadAffinityMask()` function to force the thread to run on a specific logical processor. By restricting the thread to a single core, I disable the OS ability to migrate the thread to an idle core if background applications spike CPU usage on that specific core.

Table 2: Runtime Results for $n = 30,000$

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
30,000	1	26.719472	30.264519	28.771548	28.585180
30,000	2	24.668940	28.975546	26.061706	26.568731
30,000	4	25.822573	22.863536	26.422760	25.036290
30,000	8	20.067161	20.716201	20.974003	20.585788
30,000	16	19.059442	17.447203	18.939931	18.482192
30,000	32	18.556236	18.984613	19.051165	18.864005
30,000	64	17.291712	16.436813	17.182956	16.970494

Table 3: Runtime Results for $n = 40,000$

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
40,000	1	100.762865	80.177616	80.962808	87.301096
40,000	2	56.645065	57.622343	55.039595	56.435668
40,000	4	50.919644	51.736744	52.226040	51.627476
40,000	8	43.891709	43.526415	41.273156	42.897093
40,000	16	39.686137	38.935961	39.335844	39.319314
40,000	32	39.011227	40.411656	37.879344	39.100742
40,000	64	38.022790	37.345439	36.907860	37.425363

Do you think you can now achieve $n = 50,000$ and even $n = 100,000$? Try it to see if you can. If you were able to do so, why do you think you can now do it? If not yet, why do you think you still can not?

Even though I drastically reduced the computation time by distributing the workload, I still cannot achieve $n = 50,000$ and $n = 100,000$. Similarly to the last exercise, I only was able to run until $n = 46,000$. The problem is not a computational speed but rather strict hardware memory constraints (RAM restrictions). For context, a $100,000 \times 100,000$ single-precision float matrix requires exactly 40 Gigabytes of contiguous RAM to initialize. Because this massive memory request vastly exceeds my available physical hardware, the Windows OS intervenes and terminates the `.exe` process to protect the computer from a system crash.

Graph Analysis & Observation

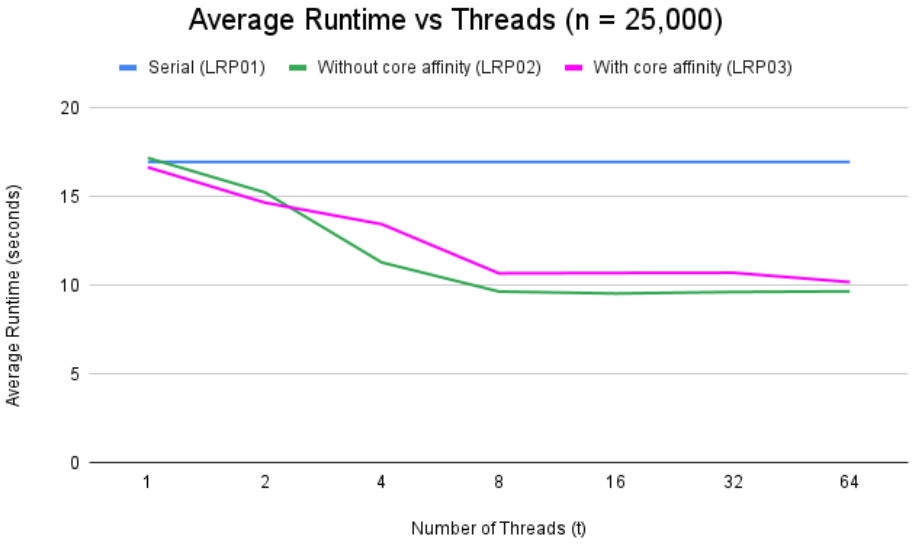

Figure 1: Average Runtime vs Threads ($n = 25,000$)

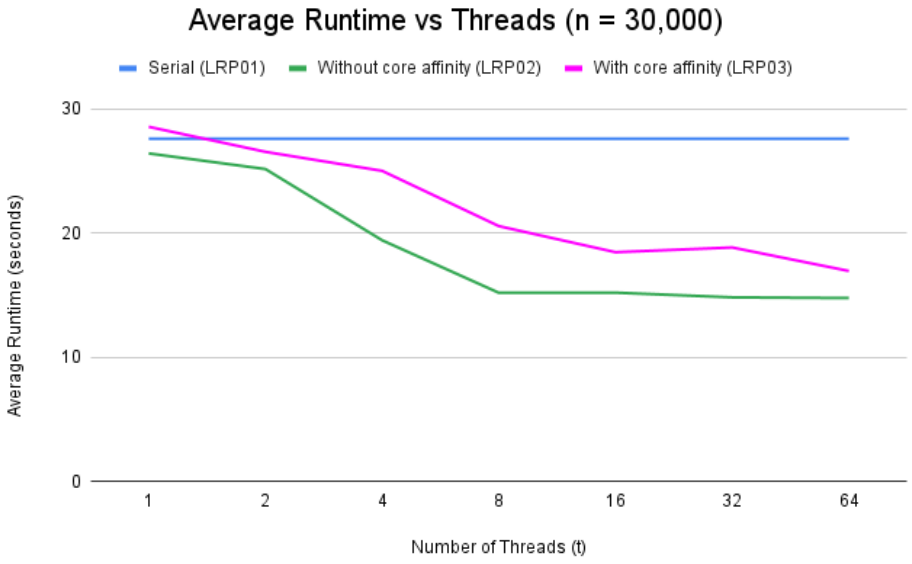

Figure 2: Average Runtime vs Threads ($n = 30,000$)

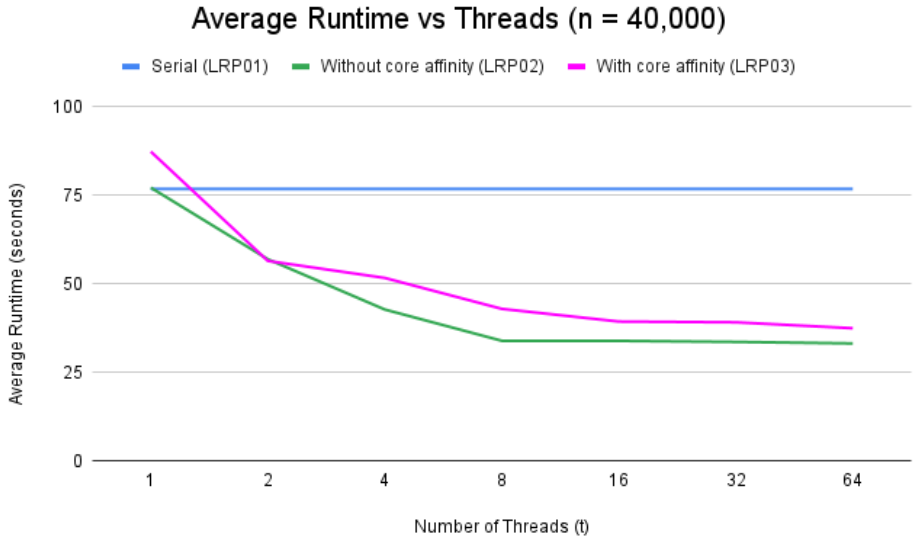

Figure 3: Average Runtime vs Threads ($n = 40,000$)

Describe in detail what you have observed.

Based on the superimposed graph, the Serial (LRP01) looks like a high, flat baseline. The threaded implementation (LRP02 and LRP03) drop sharply downward initially, proving a massive parallel speedup. However, with core affinity(LRP03), its line consistently stays slightly above the line of "without core affinity" for all $t > 1$. This proves that manually assigning threads to specific cores incurs a penalty from restricting the OS scheduler from dynamically allocating workloads to idle cores. Furthermore, because the MMT algorithm is a one-pass algorithm (each column is read only once), the rigid core pinning provides zero CPU cache locality benefits to offset the scheduling penalty.

Can we go as far as $t = n$? What about $t = n/2$? Or, $t = n/4$? Or, $t = n/8$?

NO, we cannot go as far as $t = n$. Attempting to create 25,000 individual threads, for example, would crash the program due to OS resource exhaustion and massive thread creation overhead. Even at a significantly smaller fraction like $t = n/8$, forcing over 3,000 threads to combat for execution time causes extreme thread contention. This leads to massive context switching overhead. The CPU starts spending significantly more computation time rapidly pausing threads, swapping memory registers, and managing 3,000 queues than it spends actually doing the MMT math. This is why the graph proves that performance bottoms out optimally near the physical core count, and worsens if too many threads are made.

Research Question 2

Did your average time improve compared to LRP01 and LRP02 for every t? Which t is the average time better? Statistically the same? Slower? Why?

Compared to the LRP01, the average time improved massively for all $t > 1$ because the workload was divided across multiple cores. However, compared to LRP02, the average time in LRP03 was statistically the same or slower for every t. This happens because of several reasons. One of which is because of the reduced available hardware. In LRP03, the instructions says to intentionally reserve 1 core. By restricting our threads

to only the remaining cores (in my case 7 because I have 8 cores), I mathematically gave the program less processing power than LRP02, which utilized 100% of the CPU. Connected to that is the second reason why it did not improve, OS scheduler restriction (load balancing). Modern OS are highly optimized to dynamically shift threads to idle cores. By locking threads to specific cores using affinity, we forced them to wait if their assigned core became busy, entirely removing the OS dynamic load balancer.

Research Question 3

Discuss in your own words what will happen if instead you divide X as in step (3) above, still into t submatrices, but in a manner where each submatrix is of size $n/t \times n$?

The Min-Max Transformation (MMT) formula requires the global minimum and maximum of the entire column to normalize the elements. Dividing the matrix into dimensions of $n/t \times n$ means that we are partitioning the matrix horizontally by chunks of rows. If a thread is assigned a chunk of rows, it only possesses an incomplete segment of each column. With this, the thread cannot determine the minimum and maximum of that column to perform the normalization.

To solve this problem, exactly like what I have done in the LRP02 Row Wise code, I implemented a two-phase approach. The phase 1 finds the local minimum and maximum for its segment of every column. Then, synchronization happens to gather all the local extremes to determine the true global extremes for the entire column. Then, Phase 2 happens where the MMT formula is applied to their assigned rows using newly synchronized minimum and maximum. Even though this row partitioning requires a complex synchronization phase, it is actually faster than the column-partitioning approach. Because C stores matrices in Row-Major order, assigning threads by rows ensures Spatial Cache Locality. The CPU prefetcher eliminates memory stall cache misses, making the row-wise approach faster despite the synchronization overhead.